

AWS Infrastructure Decision Utility for Terraform Generation

Strategic conclusion

Your product thesis is strong: the highest-value part is not emitting Terraform syntax, but converting vague business intent into an explicit architecture choice, a risk posture, and a cost-aware deployment plan. That aligns closely with how AWS itself frames architecture work. The AWS Well-Architected Framework is explicitly about understanding the pros and cons of architecture decisions across security, reliability, performance, cost, and sustainability, while AWS Decision Guides are organized around how to choose services for a workload rather than how to type configuration. Terraform guidance from AWS and HashiCorp follows the same pattern: codify decisions only after you have a stable abstraction, versioning model, backend, and module boundaries. ¹

The practical implication is that your system should behave like a constrained architecture advisor with a Terraform backend, not a free-form code generator. In other words, it should collect intent, derive a normalized architecture specification, validate security and cost guardrails, and only then offer Terraform. The rules below are an inference from AWS service docs, Well-Architected guidance, AWS security reference guidance, and Terraform best-practice documentation. ²

End-user input contract

The minimum viable input model should ask only for fields that materially change architecture. AWS's own compute, database, and networking decision materials consistently revolve around workload type, operational overhead, performance requirements, access patterns, and business constraints. A good contract therefore needs to capture app shape, data shape, traffic shape, exposure model, compliance posture, and budget tolerance before anything Terraform-specific is discussed. ³

A strong v1 contract is:

```
workload:
  app_type: [static_site, public_api, web_app, worker, event_processor,
internal_app, batch_job]
  packaging: [source_only, function, container, vm]
  runtime: [node, python, java, go, dotnet, other]
  statefulness: [stateless, session_state, shared_files, stateful_service]
  request_pattern: [synchronous, asynchronous, mixed]
  max_execution_time: [short_under_15m, long_running]
  environments: [dev, staging, prod]
  single_tenant_or_multi_tenant: [single, multi]
```

```

traffic:
  internet_exposed: [yes, no]
  global_users: [yes, no]
  estimated_rps: integer
  traffic_variability: [steady, diurnal, spiky, unpredictable]
  websocket_or_streaming: [yes, no]
  latency_sensitivity: [low, medium, high]

data:
  primary_data_model: [none, key_value, document, relational, object, file,
cache]
  relational_features_needed: [joins, transactions, reporting, none]
  read_write_profile: [read_heavy, write_heavy, balanced]
  data_retention: [short, medium, long]
  backups_required: [yes, no]
  disaster_recovery_tier: [none, in_region_ha, cross_region]

security_and_compliance:
  data_sensitivity: [public, internal, confidential, regulated]
  compliance: [none, soc2_like, hipaa_like, pci_like, custom]
  encryption_customer_managed: [yes, no]
  public_access_exceptions: [none, approved_public_s3, public_api, public_alb]
  secret_types: [none, app_secrets, db_credentials, api_keys]
  human_access_model: [console_ok, federated_only, break_glass_only]

networking:
  needs_vpc_private_resources: [yes, no]
  outbound_internet_needed_from_private: [yes, no]
  private_connectivity_only: [yes, no]
  on_prem_or_other_vpc_connectivity: [yes, no]

operations:
  team_ops_maturity: [low, medium, high]
  logs_and_metrics_required: [baseline, enhanced, audit_grade]
  change_frequency: [low, medium, high]

cost:
  monthly_budget_band: [minimal, constrained, moderate, flexible]
  cost_priority: [lowest_possible, balanced, performance_first]
  commitment_ok: [none, savings_plans_ok, spot_ok]

```

This is deliberately shorter than a full cloud questionnaire, but it captures the fields that most often change the AWS resource graph. For example, `max_execution_time`, `packaging`, and `traffic_variability` drive Lambda vs container vs VM decisions; `primary_data_model` and `relational_features_needed` drive DynamoDB vs RDS/Aurora; and `internet_exposed`, `global_users`, and `private_connectivity_only` drive CloudFront, API Gateway, ALB scheme, VPC design, and endpoint choices. ⁴

A second design rule is that some inputs must be declared as **mandatory blockers** before Terraform can be generated. At minimum these are: internet exposure, environments, data sensitivity, primary data model, backup requirement, and budget band. Without them, the system cannot safely decide whether resources belong in public or private subnets, whether deletion protection and Multi-AZ are required, whether WAF or private ingress is needed, or whether a lower-cost but less-featured API tier is appropriate. That is not opinionated overengineering; it is exactly the sort of criteria AWS uses when distinguishing service choices and Well-Architected tradeoffs. ⁵

Resource derivation rules

The most reliable way to derive AWS resources is to convert the input contract into an intermediate architecture spec first. The engine should not emit raw Terraform resources immediately. It should first decide a small set of architecture primitives: compute plane, ingress plane, data plane, network plane, security baseline, observability baseline, and cost-control baseline. That keeps the reasoning understandable, auditable, and explainable in plain English. This “spec first, Terraform second” model is also consistent with Terraform module-composition guidance, which recommends keeping modules flat and composing meaningful higher-level building blocks rather than generating thin wrappers around individual resources. ⁶

Compute and execution rules

Default to **AWS Lambda** when all of the following are true: the workload is event-driven or request-driven, tasks are short-lived, the service is mostly stateless, traffic is bursty or unpredictable, and the team wants minimal infrastructure management. Lambda is designed for event-driven execution, scales automatically with events, and has a maximum function timeout of 900 seconds. If a Lambda workload must access private VPC resources, attach it to private subnets; if it also needs internet egress, use a NAT path or appropriate VPC endpoints. If Lambda talks to RDS in production or under bursty concurrency, add **Amazon RDS Proxy** and **AWS Secrets Manager**. ⁷

That rule expands to the following required resources in common cases. For a public synchronous API: Lambda, IAM execution role, CloudWatch log group, API Gateway, optional custom domain in API Gateway or CloudFront, and Route 53. For asynchronous processing: Lambda plus SQS or EventBridge, and optionally Step Functions if the workflow spans multiple steps or needs orchestration state. For private data access: add VPC attachment, security groups, subnets, and either NAT or endpoints. ⁸

Default to **Amazon ECS on AWS Fargate** when the application is containerized, needs long-running processes, needs explicit CPU and memory sizing, or requires a runtime environment that is awkward in Lambda. AWS positions Fargate as serverless compute for containers and notes it is ideal for long-running applications, microservices, or batch processing, while Lambda is better for short-duration event-driven work. In v1 of your product, ECS on Fargate should be the managed-container default and EC2-backed containers should be treated as an advanced choice. ⁹

The required bundle for a public ECS service is usually: ECR repository, ECS cluster, task definition, ECS service, IAM task role and execution role, ALB or internal ALB depending on exposure, target group, listeners, security groups, CloudWatch logs, and Route 53 if there is a custom domain. If the service is internal-only, the ALB should be internal rather than internet-facing. If the service pulls from or pushes to S3 or

DynamoDB from private subnets, consider gateway endpoints to avoid unnecessary NAT dependence and cost. ¹⁰

Choose **Amazon EC2** when the workload needs operating-system level control, custom agents or drivers, unusual networking, licensing constraints, specialized instance families, or persistent processes that do not fit well into Lambda or standard container patterns. EC2 is the broadest and most customizable compute option, but it also carries the highest operational burden relative to Lambda and Fargate. If EC2 is selected for an internet-facing application, require a launch template, Auto Scaling Group, load balancer, IAM instance profile, patching/management path through Systems Manager, and CloudWatch telemetry. ¹¹

Data rules

Default to **Amazon DynamoDB** when the primary data model is key-value or document, access patterns are known up front, scale may become very high, and the application does not fundamentally depend on joins. DynamoDB is a fully managed NoSQL service with consistent single-digit millisecond performance at any scale, but it does not support relational joins and expects denormalized modeling. In practice, this means your decision utility should ask explicit questions about joins, ad hoc reporting, and relational constraints before recommending DynamoDB. ¹²

When DynamoDB is selected, the usual required resources are: a table, optional GSIs defined from access-pattern inputs, optional Streams if downstream event handling is requested, IAM policies for the workload, and a DynamoDB gateway endpoint if the workload runs inside private subnets. If data sensitivity is confidential or regulated, turn on encryption choices and backup/point-in-time recovery settings in the generated spec. ¹³

Default to **Amazon RDS or Amazon Aurora** when the workload needs SQL, joins, strict relational semantics, or a team wants a conventional relational model. AWS positions RDS as the managed relational default and Aurora as the cloud-built relational engine compatible with MySQL and PostgreSQL. For production, use Multi-AZ for high availability; for read-heavy production workloads on MySQL or PostgreSQL where you want higher read capacity and lower failover latency, Multi-AZ DB clusters are especially relevant because they create one writer and two readers across three Availability Zones. ¹⁴

When RDS or Aurora is selected, the required bundle is typically: DB subnet group, database security group, parameter group as needed, encrypted storage, backups, deletion protection for production, and Secrets Manager for credentials. If the compute plane is Lambda and concurrency may spike, add RDS Proxy; AWS explicitly recommends that for Lambda functions that make frequent short database connections or open and close many connections. ¹⁵

Always include **Amazon S3** when the app needs object storage for uploads, static assets, logs, or backup artifacts. If the use case is a static website or static asset origin, prefer **CloudFront in front of S3** and keep S3 Block Public Access enabled where possible. AWS recommends OAC for private S3 origins because it supports all S3 Regions, SSE-KMS, and dynamic requests, while S3 website hosting endpoints themselves are HTTP-only. ¹⁶

Add **Amazon EFS** only when the input explicitly requires a shared POSIX-style file system across compute nodes, such as shared uploads, legacy file-based apps, or Lambda/ECS workloads that truly need shared file semantics. Add **ElastiCache** only when the input indicates repeated hot reads, cache-aside patterns,

sessions, or latency relief for a backing store. Both are valuable, but both should be explicit second-order choices rather than defaults. ¹⁷

Ingress and delivery rules

For **public APIs**, the default choice should be **API Gateway HTTP APIs** unless the user needs features that are only present in **REST APIs**. AWS says HTTP APIs are designed with minimal features so they can be offered at a lower price, while REST APIs should be chosen when you need API keys, per-client throttling, request validation, AWS WAF integration, or private API endpoints. That rule is clean enough to encode directly into your decision engine. ¹⁸

For **HTTP web applications or container/VM services**, default to an **Application Load Balancer**. ALB supports listener rules and target groups that enable host-based and path-based routing, which is a good fit for multiple microservices behind one ingress. The key decision is not ALB versus API Gateway in the abstract; it is whether the user is publishing an API product with gateway features or exposing conventional HTTP services that need request routing to compute targets. ¹⁹

For **global sites, APIs, or static assets**, add **CloudFront** when the app has geographically distributed users, needs caching, or uses S3 as an origin. AWS documents CloudFront as a low-latency content delivery network for static and dynamic content, and the product page highlights that it can reduce transfer-out costs and improve performance. In a static-site path, the normal resource bundle is S3 bucket, CloudFront distribution, OAC, ACM certificate, and Route 53 alias records. ²⁰

Network and topology rules

A simple but important rule is that **only edge-facing components belong in public subnets**. In a typical web stack, that means internet-facing ALBs and NAT gateways. Application compute, databases, caches, and VPC-attached Lambda functions should normally live in private subnets. AWS's VPC example architecture explicitly places load balancer nodes and NAT gateways in public subnets while the servers run in private subnets and reach the internet through the NAT gateway. ²¹

If private resources need outbound internet access, add a **NAT gateway**. A NAT gateway allows instances in private subnets to connect to services outside the VPC while preventing unsolicited inbound connections. However, if most outbound traffic is going to S3 or DynamoDB, prefer **gateway VPC endpoints** because they do not require an internet gateway or NAT device and there is no additional charge for gateway endpoints. This is one of the highest-value cost/safety rules in your engine. ²²

For **Lambda attached to a VPC**, the engine should automatically force private subnet selection. AWS is explicit that connecting a Lambda function to a public subnet does not give it internet access or a public IP; if the function needs both private-resource access and public egress, it still needs a private-subnet-plus-NAT design. That is the sort of hidden AWS behavior that makes a decision utility more valuable than plain Terraform generation. ²³

For production, default to **two or more Availability Zones** for most application tiers, and to RDS Multi-AZ for production relational databases unless the user explicitly accepts lower availability. AWS repeatedly recommends multiple AZs as a reliability best practice, and RDS uses Multi-AZ to provide failover support.

²⁴

Guardrails for security, privacy, compliance, and cost

A good decision engine should attach a **non-optional security baseline** to every architecture, with stricter controls added when the input says confidential, regulated, production, or internet-exposed. The baseline should include IAM least privilege, logging, encryption, secrets handling, and change visibility. AWS IAM best practices explicitly call for least privilege, federation and temporary credentials for human access, MFA, and regular review of unused permissions. AWS CloudTrail is used for operational and risk auditing, AWS Config records configuration history, GuardDuty provides threat detection, and Security Hub gives a centralized view of security state. ²⁵

For public web workloads, add **AWS WAF** when the selected ingress supports it and the exposure warrants it. AWS WAF protects web applications and APIs against common web exploits and can be attached to CloudFront distributions, API Gateway REST APIs, and Application Load Balancers. Because HTTP APIs do not have the same REST-only feature set, a requirement for WAF directly at API Gateway becomes a routing signal toward REST APIs. ²⁶

For S3, the default should be **Block Public Access on**, with any exception forced through an explicit override field and a risk explanation. AWS states that Block Public Access helps prevent public access granted through policies or ACLs, and for static-sites it recommends CloudFront plus OAC if you want to keep all four S3 block settings enabled. Add **IAM Access Analyzer** as a validation control, especially when public access or cross-account access is in scope; AWS notes that external-access analyzers work per Region, so the engine should model them regionally. ²⁷

For secrets, default to **AWS Secrets Manager** when the app stores database credentials, API keys, or rotating application secrets. AWS's own security guidance recommends Secrets Manager for secrets, Parameter Store for simple key-value parameters, and AppConfig for advanced runtime configuration. If the engine detects relational databases, it should strongly prefer Secrets Manager automatically. ²⁸

For regulated or high-sensitivity workloads, add **KMS-backed encryption**, backup policy, deletion protection on stateful services, vulnerability scanning, and centralized security services. AWS KMS is the core managed key service; AWS Backup centralizes and automates backups; Inspector scans EC2, ECR, and Lambda for vulnerabilities; and the AWS Security Reference Architecture recommends a multi-account foundation with guardrails for stronger separation of duties and governance. If the organization has multiple teams or a serious production footprint, default the organizational scaffold to **AWS Organizations** and strongly recommend **AWS Control Tower** rather than a single-account design. ²⁹

Your cost baseline should not stop at estimates. Attach **AWS Budgets** for threshold alerting, **AWS Cost Anomaly Detection** for spend spikes, and **AWS Compute Optimizer** or Trusted Advisor hooks for post-deployment right-sizing. For estimation itself, AWS now provides both the Pricing Calculator experience and programmatic pricing interfaces, including AWS Price List APIs and Pricing Calculator API support in Billing and Cost Management tooling. That matters because your product can move from static heuristics to live estimates without abandoning an explainable rules engine. ³⁰

Terraform generation contract

Terraform should be generated only after the system has produced a stable intermediate object such as:

```
{
  "architecture_summary": {},
  "required_resources": [],
  "optional_resources": [],
  "security_controls": [],
  "cost_controls": [],
  "risks": [],
  "assumptions": [],
  "terraform_modules": [],
  "unresolved_questions": []
}
```

If `unresolved_questions` is non-empty, the system should stop before code generation. That gating behavior is essential because Terraform best practices assume deliberate module boundaries, explicit provider version management, remote state design, and clear code organization. AWS recommends a proper remote backend, and its Terraform guidance specifically favors S3 for most AWS users. Current Terraform S3 backend documentation also supports lockfiles directly with `use_lockfile = true`, and provider/version pinning should be enforced in CI. ³¹

For module boundaries, follow three rules from HashiCorp's module guidance. First, keep the module tree relatively flat and compose from the root. Second, create modules only for meaningful architectural concepts, not single-resource wrappers. Third, scope by encapsulation, privilege boundaries, and volatility. In practice, that means modules such as `network`, `api_ingress`, `ecs_service`, `lambda_api`, `rds_postgres`, `dynamodb_table`, `static_site`, and `security_baseline` are good abstractions, while modules like `single_security_group` or `one_subnet` usually are not unless they encode a broader reusable contract. ⁶

Provider configuration should live in the **root module**, while child modules declare provider requirements only. Use the standard module structure with a root module, README, outputs, and examples. Pin Terraform and AWS provider versions, and apply consistent tags through provider default tags so that cost allocation, environment ownership, and governance are visible across resources. AWS billing documentation and tagging guidance make clear that activated cost allocation tags are central to tracking spend by environment, project, or business unit. ³²

Recommended MVP ruleset

The simplest effective MVP is not “all AWS.” It is a constrained rules engine over a small but high-value service set:

- Compute: Lambda, ECS on Fargate, EC2. ³³
- Ingress: API Gateway HTTP API, API Gateway REST API, ALB, CloudFront. ³⁴
- Data: DynamoDB, RDS/Aurora, S3. ³⁵
- Async glue: SQS, EventBridge, Step Functions. ³⁶
- Security baseline: IAM, KMS, Secrets Manager, CloudTrail, Config, GuardDuty, Security Hub, WAF where supported, Access Analyzer, Backup, Inspector. ³⁷

- Cost and ops baseline: Budgets, Cost Anomaly Detection, Compute Optimizer, CloudWatch. 38

A concrete first-pass decision policy can be expressed like this:

```
if app_type == static_site:
    choose S3 + CloudFront + OAC + Route53 + ACM
    keep S3 Block Public Access on by default
    add WAF only if high-risk public exposure or compliance requires it

if app_type == public_api:
    if needs_api_keys or per_client_throttling or request_validation or
    waf_on_api_gateway or private_api:
        choose API Gateway REST API
    else:
        choose API Gateway HTTP API
    then:
        if max_execution_time == short_under_15m and packaging in [function,
source_only]:
            choose Lambda
        elif packaging == container or max_execution_time == long_running:
            choose ECS Fargate
        else:
            choose EC2 only if explicit control requirement exists

if app_type in [web_app, internal_app]:
    if packaging == container:
        choose ECS Fargate + ALB
    elif packaging == vm:
        choose EC2 + ASG + ALB
    else:
        choose Lambda only when the app is really an API/event surface rather than a
conventional long-running web tier

if primary_data_model in [key_value, document] and joins not required:
    choose DynamoDB
else if relational_features_needed in [joins, transactions, reporting]:
    choose RDS or Aurora

if data_sensitivity in [confidential, regulated] or environment includes prod:
    require private subnets for app/data tiers
    require encryption, Secrets Manager, CloudTrail, Config, GuardDuty, Security
Hub, backups
    require deletion protection on production databases
    require least-privilege IAM
    prefer multi-account landing zone

if outbound_internet_needed_from_private == yes:
```



```

    add NAT gateway
    but replace NAT path with gateway endpoints for S3/DynamoDB when applicable

if global_users == yes:
    add CloudFront

if team_ops_maturity == low:
    bias toward Lambda, Fargate, DynamoDB, managed controls
else:
    allow EC2 and deeper customization

```

The thresholds and defaults in that MVP are product choices rather than AWS laws, but they are grounded in AWS's documented service boundaries. The result is a system that can answer the hard questions your users actually have: what to use, what must be private, which controls are missing, what the cheapest safe option looks like, and only then which Terraform modules to emit. ³⁹

¹ <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

<https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

² <https://docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/introduction.html>

<https://docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/introduction.html>

³ <https://docs.aws.amazon.com/compute-on-aws-how-to-choose/>

<https://docs.aws.amazon.com/compute-on-aws-how-to-choose/>

⁴ ⁷ ³³ ³⁹ <https://docs.aws.amazon.com/decision-guides/latest/fargate-or-lambda/fargate-or-lambda.html>

<https://docs.aws.amazon.com/decision-guides/latest/fargate-or-lambda/fargate-or-lambda.html>

⁵ ¹⁸ ³⁴ <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>

⁶ <https://developer.hashicorp.com/terraform/language/modules/develop>

<https://developer.hashicorp.com/terraform/language/modules/develop>

⁸ <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>

<https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>

⁹ <https://docs.aws.amazon.com/decision-guides/latest/containers-on-aws-how-to-choose/choosing-aws-container-service.html>

<https://docs.aws.amazon.com/decision-guides/latest/containers-on-aws-how-to-choose/choosing-aws-container-service.html>

¹⁰ <https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>

<https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html>

¹¹ <https://docs.aws.amazon.com/ec2/>

<https://docs.aws.amazon.com/ec2/>

¹² ³⁵ <https://docs.aws.amazon.com/lambda/latest/dg/ddb-rds-database-decision.html>

<https://docs.aws.amazon.com/lambda/latest/dg/ddb-rds-database-decision.html>

- 13 <https://docs.aws.amazon.com/vpc/latest/privatelink/gateway-endpoints.html>
<https://docs.aws.amazon.com/vpc/latest/privatelink/gateway-endpoints.html>
- 14 <https://docs.aws.amazon.com/rds/>
<https://docs.aws.amazon.com/rds/>
- 15 <https://docs.aws.amazon.com/lambda/latest/dg/services-rds.html>
<https://docs.aws.amazon.com/lambda/latest/dg/services-rds.html>
- 16 <https://aws.amazon.com/s3/>
<https://aws.amazon.com/s3/>
- 17 <https://docs.aws.amazon.com/efs/latest/ug/whatisefs.html>
<https://docs.aws.amazon.com/efs/latest/ug/whatisefs.html>
- 19 <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/application-load-balancers.html>
<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/application-load-balancers.html>
- 20 <https://docs.aws.amazon.com/cloudfront/>
<https://docs.aws.amazon.com/cloudfront/>
- 21 <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html>
<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html>
- 22 <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>
<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>
- 23 <https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-internet.html>
<https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-internet.html>
- 24 https://aws.github.io/aws-elb-best-practices/reliability/workload_architecture/
https://aws.github.io/aws-elb-best-practices/reliability/workload_architecture/
- 25 37 <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
<https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- 26 <https://aws.amazon.com/waf/>
<https://aws.amazon.com/waf/>
- 27 <https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html>
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html>
- 28 <https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html>
<https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html>
- 29 <https://docs.aws.amazon.com/kms/>
<https://docs.aws.amazon.com/kms/>
- 30 38 <https://aws.amazon.com/documentation-overview/budgets/>
<https://aws.amazon.com/documentation-overview/budgets/>
- 31 <https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/backend.html>
<https://docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/backend.html>
- 32 <https://developer.hashicorp.com/terraform/language/modules/develop/providers>
<https://developer.hashicorp.com/terraform/language/modules/develop/providers>

³⁶ <https://docs.aws.amazon.com/sqs/>
<https://docs.aws.amazon.com/sqs/>